Software Project Organization: A Repository Layout for Autonomous Coding Loops

Organizing specifications, prompts, knowledge, and state so an autonomous agent knows where to read and where to write

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

An autonomous coding loop runs the same agent against a repository over and over until it satisfies a goal. Loop reliability depends on repository structure as much as on model capability: an agent that cannot find the current specification will invent one, and an agent with no fixed place to record progress will repeat work it already finished. This paper identifies a Software Project Organization (SPO), a repository layout organized around the software lifecycle rather than around the programming language. The SPO gives every artifact class a single home, separates three kinds of persistent context that conventional projects blur together (domain knowledge, agent memory, and loop state), defines which files each agent role may read, write, and never touch, and specifies an exclusion policy that keeps secrets and ephemeral run data out of version control. The layout is a working standard rather than a validated one; the closing section lists what still needs testing.

1. Introduction

Conventional repository layouts assume a human reader. A developer who cannot find the requirements document asks a teammate, checks a wiki, or reconstructs the requirement from the code. An agent running inside a loop has none of those options. It sees a file tree, a prompt, and whatever context the harness loads, and it acts on what it finds.

That constraint changes what a good layout optimizes for. Language-idiomatic structures such as `src/`, `lib/`, and `tests/` organize code by role in the build, which serves compilers and package managers well. They say nothing about where the specification lives, where the prompt that produced that specification lives, or where the agent should record that it finished task 14 of 30. Those artifacts end up scattered across a wiki, a ticket tracker, a chat history, and a developer's local notes, which is exactly the material an autonomous loop needs and cannot reach.

The pattern emerging in agentic development inverts the usual priority. Code becomes one artifact among several rather than the organizing center. Specifications, prompts, architectural decisions, tests, and accumulated project knowledge sit alongside the source and receive the same versioning discipline. SPO formalizes that inversion into a directory structure, an artifact lifecycle, and a permission model.

The immediate motivation is a loop pattern sometimes called a Ralph loop: an agent invoked repeatedly with a stable prompt, allowed to pick its next task from a queue, make a change, and exit, with the harness restarting it until the queue empties. The pattern is simple to run and unforgiving of disorganization, which makes it a useful design target. A layout that works for a loop of this kind will also serve a supervised workflow where a human approves each step.

2. Design principles

Four principles drive the layout.

Organize by lifecycle, not by language. Top-level directories correspond to stages of development: goals, requirements, design, build, verify, release, and knowledge. An agent asked to write a specification does not need to know that the project is

Python; it needs to know that specifications live in `docs/` and that the requirements it must satisfy live next to them. Language-specific structure stays inside `src/`, where it belongs.

Give every artifact exactly one home. Ambiguity about where a file belongs produces duplicates, and duplicate specifications diverge. Each artifact class in SPO has one directory, and the agent instruction file records the mapping so no agent has to guess.

Separate permanent knowledge from learned memory and from current state. Section 4 develops this distinction; it is the part of SPO that has no analogue in a conventional project layout.

Make exclusion explicit. Generated output, logs, secrets, and loop state either belong in version control or they do not, and the decision should be visible in `.gitignore` rather than left to whoever runs `git add`.

3. Repository layout

The tree below is a starting template of roughly seventy entries. Projects that run long agentic loops tend to grow past that count as decision records and prompt variants accumulate.

```
project-root/
|
+-- README.md                  # Project overview for humans
+-- PROJECT.md                 # Mission, goals, success criteria
+-- ROADMAP.md                 # Milestones and sequencing
+-- CHANGELOG.md
+-- CONTRIBUTING.md
+-- LICENSE
+-- AGENTS.md                  # Standing instructions for AI agents
+-- .gitignore
+-- .gitattributes
+-- .editorconfig
+-- .env.example               # Placeholders only, never real values
|
+-- docs/                      # Human- and agent-readable specifications
|   +-- vision.md
|   +-- requirements.md
|   +-- specification.md
|   +-- architecture.md
|   +-- api-spec.md
|   +-- deployment.md
|   +-- coding-standards.md
|   +-- project-layout.md
|   +-- test-plan.md
|   +-- user-guide.md
|   +-- developer-guide.md
|   +-- diagrams/
|   |   +-- architecture.drawio
|   |   +-- data-model.drawio
|   |   +-- workflow.drawio
|   +-- decisions/             # Architecture Decision Records
|       +-- ADR-template.md
|       +-- ADR-001-loop-runner.md
|       +-- ADR-002-state-format.md
|
+-- prompts/                   # Versioned, modular prompt library
|   +-- system/
|   |   +-- architect.md
|   |   +-- coder.md
|   |   +-- reviewer.md
|   |   +-- tester.md
|   +-- planning/
|   +-- specification/
```

```
|   +-- implementation/
|   +-- testing/
|   +-- documentation/
|   +-- release/
|
+-- workflows/                # Loop definitions that compose prompts
|   +-- plan-loop.yaml
|   +-- design-loop.yaml
|   +-- implement-loop.yaml
|   +-- review-loop.yaml
|   +-- test-loop.yaml
|   +-- release-loop.yaml
|
+-- knowledge/                # Durable domain truth
|   +-- glossary.md
|   +-- domain-model.md
|   +-- business-rules.md
|   +-- references.md
|
+-- memory/                   # What the agents have learned building this
|   +-- project-memory.md
|   +-- completed-tasks.md
|   +-- lessons-learned.md
|   +-- known-issues.md
|
+-- state/                    # Current position of the loop
|   +-- backlog.json
|   +-- current-task.json
|   +-- next-task.json
|   +-- iteration-summary.md
|
+-- src/
|   +-- api/
|   +-- services/
|   +-- agents/
|   +-- models/
|   +-- utils/
|   +-- config/
|
+-- tests/                    # Mirrors src/
|   +-- unit/
|   +-- integration/
|   +-- acceptance/
|   +-- regression/
|   +-- fixtures/
|
+-- scripts/
|   +-- bootstrap.sh
|   +-- build.sh
|   +-- test.sh
|   +-- lint.sh
|   +-- release.sh
|
+-- output/                   # Everything the loop generates
|   +-- generated-code/
|   +-- generated-docs/
|   +-- reports/
|   +-- releases/
|
+-- logs/
+-- temp/
+-- secrets/
    +-- README.md             # How to populate; checked in
    +-- local.env             # Ignored
```

Three directories deserve comment.

`AGENTS.md` carries the standing instructions every agent receives: the layout contract, the coding standards it must follow, and the permission table from Section 5. Keeping those instructions in one versioned file at the repository root means a change to the working agreement is a reviewable diff.

`prompts/` treats prompts as source. Splitting them by lifecycle stage, with a `system/` subdirectory for role definitions, lets a workflow compose a run from a role prompt plus a stage prompt instead of maintaining one monolithic instruction per task. Prompt changes then show up in the history alongside the code they produced, which matters when a regression traces back to an edited prompt rather than to an edited function.

`output/` isolates generated artifacts from hand-maintained ones. An agent that writes a report into `docs/` has polluted the specification directory; an agent that writes it into `output/reports/` has not.

4. Knowledge, memory, and state

Conventional projects keep documentation and leave everything else in a developer's head. Agentic projects cannot, and the three kinds of context an agent needs behave differently enough that mixing them causes trouble.

Directory	Contents	Changes when	Lifetime	In version control
`knowledge/`	Domain facts, glossary, business rules, external references	The business or domain changes	Outlives the project	Yes
`memory/`	What agents learned while building: solved problems, rejected approaches, recurring failure modes	The agents learn something worth keeping	Life of the project	Yes
`state/`	Current task, backlog, iteration summary	Every loop iteration	Life of a run	Partly; see Section 6

The distinction earns its keep in two ways. It tells an agent where to write, so a lesson about a flaky integration test lands in `memory/lessons-learned.md` rather than being appended to the glossary. It also controls what enters the context window: a planning agent loads `knowledge/` and `state/backlog.json` and can skip the rest, while a debugging agent loads `memory/known-issues.md` first.

Most projects skip the memory directory. Its absence shows up as an agent solving in iteration 40 the same problem it solved in iteration 12. Writing a short entry after each substantive change costs one file append and saves a full re-derivation.

5. Artifact lifecycle and agent permissions

The layout describes where artifacts live. A standard also has to describe how they flow and who may modify them. The SPO lifecycle runs:

Vision -> Requirements -> Specification -> Architecture -> Tasks -> Code -> Tests -> Review -> Release -> Knowledge update

Each stage consumes upstream artifacts and produces one of its own. The knowledge update closes the loop by feeding what the run taught back into `memory/` and, where the domain understanding itself changed, into `knowledge/`.

Stage	Produces	Reads	Role
Vision	`PROJECT.md`, `docs/vision.md`	Human input	Human, agent assists
Requirements	`docs/requirements.md`	Vision, `knowledge/`	Analyst
Specification	`docs/specification.md`, `docs/api-spec.md`	Requirements, `knowledge/`	Architect
Architecture	`docs/architecture.md`, `docs/decisions/`	Specification	Architect
Tasks	`state/backlog.json`	Architecture, specification	Planner
Code	`src/`	Task, specification, coding standards	Coder
Tests	`tests/`	Task, `docs/test-plan.md`	Tester
Review	`output/reports/`	Diff, coding standards	Reviewer
Release	`CHANGELOG.md`, `output/releases/`	Test results, review report	Release agent
Knowledge update	`memory/`	Iteration summary	Any role

The permission model matters more than the directory names. Each role gets a read set, a write set, and a forbidden set. A coder agent reads the specification and writes `src/` and `tests/`; it never edits `docs/specification.md`, because an agent that can rewrite the specification to match its implementation will always report success. A reviewer agent reads everything and writes only reports. A planner agent owns `state/backlog.json` and touches no source file.

6. Secrets and repository hygiene

`.gitignore` is the front line for keeping credentials out of the history. The policy has three parts: keep real secrets in ignored files, check in a placeholder template so a new contributor knows what to populate, and ignore everything the loop regenerates.

```
# Secrets
.env
.env.*
!.env.example
secrets/
*.pem
*.key
*.p12
*.pfx

# Python
__pycache__/
*.pyc
.venv/

# Node
node_modules/

# Build output
dist/
build/
coverage/

# Logs
logs/
*.log

# Temporary files
temp/
tmp/
.cache/

# IDE
.vscode/
.idea/

# Operating system
.DS_Store
Thumbs.db

# Agent output
output/generated-code/
output/reports/

# Runtime loop state
state/current-task.json
state/session.json
```

The negated pattern `!.env.example` re-includes the template after the `.env.*` rule excludes it, so contributors get the placeholder file and nobody commits the populated one. And `.gitignore` only affects untracked files; a credential already committed stays in the history until someone rewrites it, and it should be treated as compromised regardless.

The split inside `state/` follows the same logic as the rest of the layout. Long-lived artifacts stay in version control: `docs/`, `prompts/`, `workflows/`, `knowledge/`, `memory/`, and the backlog. Per-run scratch data does not: the current task pointer, session files, logs, and generated output. Committing the current task pointer produces a commit per iteration and a diff history no human will read.

Two companion files support the same goal. `.gitattributes` normalizes line endings and marks generated paths so diffs stay readable across platforms, and `.editorconfig` keeps indentation consistent when agents and humans edit the same files from different tools.

7. Running the loop

A single iteration under SPO proceeds as follows.

1. The harness starts the agent with the system prompt from `prompts/system/`, the stage prompt from the matching lifecycle directory, and `AGENTS.md`.
2. The agent reads `state/current-task.json`. If it is empty, it pulls the highest-priority item from `state/backlog.json` and writes it there.
3. The agent loads the artifacts its role permits: the specification for a coder, the test plan for a tester, `memory/known-issues.md` when the task is a bug fix.
4. The agent makes its change inside its write set and runs `scripts/test.sh`.
5. On success, the agent appends to `memory/completed-tasks.md`, updates `state/iteration-summary.md`, clears `state/current-task.json`, and exits. On failure, it records the failure in `memory/known-issues.md` and either retries or returns the task to the backlog.
6. The harness restarts the agent, which finds a clean state file and takes the next task.

Every step in that sequence names a specific file, and that property makes the loop repeatable; an agent restarted with no conversational memory reconstructs its position from disk in a fixed number of reads.

`workflows/` holds the declarative definitions that drive this sequence, one per lifecycle stage, naming the prompts to compose, the artifacts to read, the write set, and the exit condition.

8. Limitations

SPO is captured from design reasoning, not a validated standard. Several gaps deserve attention before adopting it broadly.

The layout carries overhead that small projects will not repay. A two-week script does not need eleven documents in `docs/` and six loop definitions; the honest minimum is `PROJECT.md`, `AGENTS.md`, `docs/specification.md`, `state/backlog.json`, `memory/`, and the source and test trees.

Memory quality degrades without curation. An agent appending to `memory/lessons-learned.md` on every iteration produces a file that eventually exceeds the context budget and dilutes the useful entries. Some compaction policy is needed, whether periodic summarization or an explicit entry limit per file.

The permission model is unenforced, as noted in Section 5.

No empirical evaluation exists. The claims here concern developer and agent behavior and are testable: run comparable projects with and without the standard and measure rework, duplicate effort, and human intervention count.

VIRGINIA TECH

Appendix A: Repository Inventory

Type	File or Directory	Typical Contents	Example
Root files	`README.md`	The front door. What the software does, who it serves, how to install it, how to run it, and where deeper documentation lives. Useful to humans and agents meeting the repository for the first time.	"Imports customer invoices, validates them, and posts approved records to the accounting API. Run `./scripts/bootstrap.sh`, then `./scripts/test.sh`."
Root files	`PROJECT.md`	The authoritative charter: mission, problem statement, target users, objectives, non-goals, success measures, scope boundaries, guiding principles. Agents consult it before proposing major changes.	"Goal: cut manual invoice processing by 80%. Non-goal: replace the accounting system. Success: 95% of valid invoices process without human intervention."
Root files	`ROADMAP.md`	The expected sequence of major outcomes, not a task list. Phases, milestones, dependencies, target releases, acceptance conditions, known uncertainties.	"Phase 1: parse PDF invoices. Phase 2: validate vendors and totals. Phase 3: submit approved invoices. Phase 4: human-review dashboard."
Root files	`CHANGELOG.md`	A human-readable record of meaningful changes by release, grouped under Added, Changed, Fixed, Deprecated, Removed, and Security. Usually tracks semantic versions.	"0.3.0: added duplicate-invoice detection; fixed rounding errors in tax calculations."
Root files	`LICENSE`	The legal terms under which others may use, modify, or distribute the software. Standard license text rather than custom prose.	MIT, Apache 2.0, GPL, or a proprietary internal-use license.
Root files	`.gitignore`	Rules telling Git which untracked files to skip: secrets, local environments, dependencies, build output, logs, caches, temporary files, agent runtime state. Does not remove files already committed.	`.env`, `secrets/`, `node_modules/`, `.venv/`, `coverage/`, `logs/`, `temp/`
Root files	`.gitattributes`	Repository-level Git behavior: line-ending normalization, binary file identification, generated-file diff suppression, merge configuration.	`* text=auto`, `*.sh text eol=lf`, `*.png binary`, `package-lock.json -diff`
Root files	`.editorconfig`	Editor-independent formatting basics: indentation, character encoding, line endings, trailing whitespace, final newlines. Reduces accidental formatting churn between humans and agents.	"UTF-8, LF line endings, four spaces for Python, two spaces for YAML, insert a final newline."
Root files	`.env.example`	A committed template listing every required environment variable without real credentials. Comment each variable with its format and whether it is required.	`ACCOUNTING_API_URL=https://api.example.com`, `ACCOUNTING_API_KEY=replace-me`
Root files	`AGENTS.md`	Operational instructions for coding agents: how they work, required reading, commands, quality gates, architectural constraints, files they may and may not change, and how they report completion.	See Appendix A.
Core docs	`docs/vision.md`	A narrative description of the desired future state: the user problem, why it matters, what a successful experience feels like, and the product principles behind decisions. Inspiring but concrete.	"A finance clerk drops an invoice into the system and receives a validated result in under one minute."
Core docs	`docs/requirements.md`	Functional and nonfunctional requirements with stable identifiers: user needs, constraints, edge cases, priorities, dependencies, acceptance criteria. States what must happen without prescribing implementation.	`REQ-F-014: The system shall reject an invoice whose vendor is inactive. REQ-NF-003: Processing shall complete within 30 seconds for 95% of invoices.`
Core docs	`docs/specification.md`	The behavioral contract derived from the requirements: workflows, inputs, outputs, validation rules, error conditions, state transitions, interfaces, testable acceptance scenarios. Often the primary input to the coding loop.	"A duplicate invoice number for the same vendor stops processing and returns error code `DUPLICATE_INVOICE`."
Core docs	`docs/architecture.md`	The high-level technical design: components, boundaries, data flow, deployment topology, external systems, trust boundaries, technology choices, architectural constraints. Links to individual ADRs for decision history.	"The API receives upload requests, the processing service validates documents, and a queue isolates accounting-system submissions."
Core docs	`docs/api-spec.md`	The contract for internal or external APIs: endpoints, operations, authentication, schemas, status codes, errors, pagination, retries, idempotency, examples. An OpenAPI file can supplement or replace parts of it.	`POST /v1/invoices` with request schema, `202 Accepted`, validation-error response, and idempotency-key behavior.
Core docs	`docs/deployment.md`	How to deploy, configure, upgrade, roll back, and observe the application: environments, dependencies, health checks, database migrations, secret injection, incident considerations.	"CI runs production deployments. Migrations execute before application rollout. Roll back when the health-check failure rate exceeds 5%."

Type	File or Directory	Typical Contents	Example
Core docs	`docs/coding-standards.md`	Project conventions beyond automatic formatting: naming, module boundaries, error handling, logging, dependency choices, comments, typing, security, code-review expectations.	"Domain services must not import HTTP framework modules. All public Python functions require type annotations."
Core docs	`docs/project-layout.md`	Project organization conventions identifying where the project files are located, how the structure of the project is organized	"The requirements, specifications, prompts, code, and tests are organized into a standard project layout."
Core docs	`docs/test-plan.md`	The verification strategy: test levels, risk areas, environments, test data, coverage expectations, entry and exit criteria, ownership, automation, required evidence.	"Every requirement marked Critical carries at least one acceptance test. Payment submission requires integration tests against the sandbox API."
Core docs	`docs/user-guide.md`	End-user instructions organized around real tasks: setup, normal workflows, examples, screenshots where useful, error recovery, terminology, frequently asked questions.	"Uploading an invoice," "Resolving a validation failure," and "Reviewing prior submissions."
Core docs	`docs/developer-guide.md`	Onboarding and daily development instructions: repository structure, local setup, commands, branching, debugging, testing, adding features, database changes, pull-request expectations.	"Run `bootstrap.sh`, copy `.env.example` to `.env`, start the API with `make dev`, and run unit tests before committing."
Diagrams	`docs/diagrams/architecture.drawio`	An editable system-context or container diagram showing major applications, services, users, data stores, external systems, and communication paths. Labels match the architecture document.	User to web application to invoice API to processing service to accounting API.
Diagrams	`docs/diagrams/database.drawio`	An editable data-model diagram showing entities, important fields, relationships, cardinality, and ownership. Illuminates the model rather than reproducing every implementation detail.	`Vendor 1:many Invoice`; `Invoice 1:many ValidationResult`; unique vendor and invoice-number constraint.
Diagrams	`docs/diagrams/workflow.drawio`	A visual representation of the business or agentic workflow, including decisions, loops, approvals, failure routes, and completion states.	Read task, implement change, run checks, review result, revise or complete.
Decision records	`docs/decisions/ADR-001.md`	The first recorded architectural decision. Each ADR carries title, status, date, context, decision, alternatives, consequences, and follow-up actions. ADR numbers stay stable.	"Use PostgreSQL for transactional storage because invoice and audit records require relational integrity."
Decision records	`docs/decisions/ADR-002.md`	A subsequent architectural decision in the same format. ADRs capture decisions the team would otherwise re-debate or struggle to infer from the code.	"Use a queue between validation and accounting submission to isolate external API outages."
Decision records	`docs/decisions/ADR-template.md`	A blank, standardized ADR that agents and contributors copy when documenting a new technical decision. Brief guidance sits under each heading.	Headings for Status, Context, Decision, Alternatives, Consequences, Security, and Follow-up.
Agent roles	`prompts/system/architect.md`	Durable instructions for the architecture role: responsibilities, required inputs, expected outputs, decision criteria, restrictions, review checklist.	"Read the requirements and existing ADRs. Produce component boundaries and identify unresolved decisions. Do not write application code."
Agent roles	`prompts/system/coder.md`	Instructions for an implementation agent: how it selects work, reads specifications, changes code, writes tests, handles ambiguity, runs checks, and reports completion.	"Implement only the active task. Preserve public interfaces unless the specification authorizes a change. Run formatting, static analysis, and relevant tests."
Agent roles	`prompts/system/reviewer.md`	Instructions for an independent review role covering correctness, requirement traceability, architecture, security, maintainability, test sufficiency, and finding format.	"Do not modify code during review. Classify findings as blocking, major, or minor and cite the relevant file and requirement."
Agent roles	`prompts/system/tester.md`	Instructions for generating and executing verification work: test-design techniques, required test levels, expected evidence, defect reporting, independence from implementation assumptions.	"Derive test cases from requirements and edge cases. Include at least one negative case for every validation rule."
Planning prompts	`prompts/planning/create-roadmap.md`	Converts the charter and requirements into milestones and outcome-oriented phases. Keeps the agent from producing an excessively detailed task dump.	"Group work by demonstrable capability. Identify dependencies, risks, and the exit condition for each milestone."
Planning prompts	`prompts/planning/decompose-feature.md`	Turns a feature or specification into small, independently verifiable implementation tasks.	"Create tasks that fit within one agent iteration and give each task an objective, affected files, dependencies, and acceptance checks."
Planning prompts	`prompts/planning/select-next-task.md`	Rules for choosing the next task in an automated loop based on priority, prerequisites, risk, and current repository state.	"Choose the highest-priority unblocked task. Prefer risk reduction over cosmetic improvements."
Spec prompts	`prompts/specification/create-requirements.md`	Converts goals, interviews, or rough notes into uniquely identified, testable requirements.	"Separate functional requirements from quality attributes. Flag assumptions and unresolved questions rather than inventing answers."

Type	File or Directory	Typical Contents	Example
Spec prompts	`prompts/specification/create-specification.md`	The primary prompt for transforming approved requirements into an implementation-ready behavioral specification.	"Define inputs, outputs, invariants, error cases, state changes, permissions, and acceptance scenarios for every in-scope requirement."
Spec prompts	`prompts/specification/review-specification.md`	A review prompt for detecting ambiguity, inconsistency, missing edge cases, untestable language, and conflicts with project goals.	"Identify every use of vague terms such as 'quickly,' 'appropriate,' or 'normally' and propose measurable replacements."
Implementation prompts	`prompts/implementation/implement-task.md`	The central coding-loop prompt. Tells the agent how to read the active task, inspect existing code, implement the smallest correct change, run checks, update state, and stop.	"Do not begin another backlog task. Continue revising the active task until all required checks pass or a genuine blocker is documented."
Implementation prompts	`prompts/implementation/refactor.md`	Structural improvement without changing externally observable behavior, including test protection and scope control.	"Establish passing characterization tests before modifying behaviorally sensitive code."
Implementation prompts	`prompts/implementation/fix-defect.md`	A defect-resolution workflow emphasizing reproduction, root cause, regression testing, minimal correction, and verification.	"First create a failing test that reproduces the issue. Do not merely suppress the reported symptom."
Testing prompts	`prompts/testing/create-tests.md`	Generates tests from requirements, specifications, implementation changes, risk, and prior defects.	"Cover happy paths, boundaries, invalid values, permissions, concurrency risks, and expected failure behavior."
Testing prompts	`prompts/testing/run-verification.md`	Defines the sequence of automated checks, how failures are diagnosed, and what evidence is recorded.	"Run targeted tests first, then the complete test suite. Do not mark the task complete when a required check is skipped."
Testing prompts	`prompts/testing/adversarial-review.md`	A red-team prompt for breaking the implementation through malformed input, unusual state, security abuse, and failure injection.	"Test duplicate submissions, partial outages, oversized inputs, authorization bypass attempts, and retries after timeouts."
Doc prompts	`prompts/documentation/update-docs.md`	Identifies and updates documentation affected by a code or behavior change.	"Update public behavior, configuration, API examples, operational instructions, and requirement traceability where applicable."
Doc prompts	`prompts/documentation/create-user-guide.md`	Generates task-oriented end-user documentation from an approved specification and working software.	"Use user language rather than class or database names. Include expected outcomes and recovery steps."
Doc prompts	`prompts/documentation/create-release-notes.md`	Converts completed changes into concise, audience-appropriate release notes.	"Describe user-visible impact, upgrade actions, fixed defects, known limitations, and breaking changes."
Release prompts	`prompts/release/prepare-release.md`	The pre-release checklist: versioning, tests, security scans, documentation, migrations, packaging, approval evidence.	"Verify a clean working tree, passing pipeline, updated changelog, migration rehearsal, rollback procedure, and release artifact checksum."
Release prompts	`prompts/release/release-review.md`	An independent go/no-go assessment based on release criteria and unresolved risk.	"Return `GO`, `NO-GO`, or `GO WITH CONDITIONS`, followed by evidence and outstanding risks."
Workflows	`workflows/plan-loop.yaml`	The planning workflow: inputs, assigned role, prompt, allowed files, completion checks, maximum iterations, outputs.	Read `PROJECT.md` and requirements, run the planning prompt, update the roadmap and backlog, request review.
Workflows	`workflows/design-loop.yaml`	The architecture and specification refinement workflow, alternating between architect and reviewer until design criteria pass.	Generate architecture, review against requirements, revise, record important decisions as ADRs.
Workflows	`workflows/implement-loop.yaml`	The Ralph-style iterative coding loop: task selection, implementation, testing, review, retry conditions, stop conditions, budgets, state updates.	Select one task, implement, test, review, correct failures, commit completion evidence, select another task.
Workflows	`workflows/review-loop.yaml`	The independent code and artifact review workflow: reviewer context, severity rules, output format, and whether the reviewer may edit files.	Review the diff and related requirements, emit findings, return to implementation while blocking findings remain.
Workflows	`workflows/test-loop.yaml`	The verification workflow coordinating test creation, execution, defect classification, correction, and reruns.	Generate missing tests, run the targeted suite, run the complete suite, record results, reopen the task on failure.
Workflows	`workflows/release-loop.yaml`	The controlled release workflow: release criteria, approvals, versioning, packaging, deployment, smoke tests, rollback triggers, post-release updates.	Prepare candidate, verify, approve, deploy, smoke test, update changelog and release status.

Type	File or Directory	Typical Contents	Example
Knowledge	`knowledge/glossary.md`	Canonical definitions for domain, product, technical, and organizational terms. Notes prohibited synonyms where terminology must stay precise.	"Invoice date: the date printed by the vendor. Received date: the date the platform accepted the upload."
Knowledge	`knowledge/domain-model.md`	A conceptual description of domain entities, value objects, relationships, lifecycle rules, invariants, and significant events. Broader than the physical database design.	"An Invoice belongs to one Vendor and may carry several validation findings. A submitted invoice is immutable."
Knowledge	`knowledge/business-rules.md`	Centralized business policies with stable identifiers, rationale, examples, exceptions, ownership, and effective dates.	`BR-017: An invoice over $25,000 requires approval from two authorized reviewers.`
Knowledge	`knowledge/references.md`	Curated authoritative sources: standards, internal policies, vendor documentation, research, external contracts. Records why each source matters and its version or retrieval date.	"Accounting API v3 documentation: authoritative source for submission schemas and retry behavior."
Agent memory	`memory/project-memory.md`	A concise summary of repository knowledge that is hard to rediscover but does not belong in a requirement or ADR. Should not become an uncontrolled diary.	"Validation errors use machine-readable codes. All external API calls pass through `AccountingGateway`."
Agent memory	`memory/completed-tasks.md`	A historical index of completed tasks with identifier, outcome, related commits or files, verification evidence, and follow-up work.	"TASK-042 completed: duplicate detection added; 18 tests passed; follow-up TASK-047 created for reporting."
Agent memory	`memory/lessons-learned.md`	Reusable insights from defects, failed approaches, incidents, or unexpectedly difficult work: problem, cause, lesson, future rule.	"Mocking the entire accounting client concealed serialization defects. Future integration tests must verify the emitted JSON."
Agent memory	`memory/known-issues.md`	Accepted defects, technical debt, operational limitations, and unresolved risks, with severity, impact, workaround, owner, and planned resolution.	"KI-009: large scanned PDFs can exceed the processing timeout. Workaround: split documents before upload."
Runtime state	`state/backlog.json`	The machine-readable work queue. Each task carries ID, title, objective, status, priority, dependencies, source requirements, acceptance criteria, attempts, assigned workflow.	`{"id":"TASK-042","status":"ready","priority":1,"depends_on":["TASK-018"]}`
Runtime state	`state/current-task.json`	The single task the loop is processing, with attempt number, start time, working assumptions, affected areas, and latest check results. Usually Git-ignored when it holds purely runtime data.	`{"task_id":"TASK-042","attempt":3,"phase":"testing","last_result":"2 tests failing"}`
Runtime state	`state/next-task.json`	A staging record for the task chosen to run next. Stores the selection rationale and keeps two agents from claiming the same work.	`{"task_id":"TASK-043","reason":"highest-priority unblocked security task"}`
Runtime state	`state/iteration-summary.md`	A readable account of the latest loop iteration: objective, files changed, commands run, results, unresolved problems, recommended next action.	"Implemented vendor-status validation. Unit tests pass; sandbox integration failed because credentials were unavailable."
Source code	`src/api/`	Transport-layer code: routes, controllers, request parsing, response serialization, authentication hooks, API-specific error translation. Business logic lives elsewhere.	`invoice_routes.py`, `request_schemas.py`, `error_handlers.py`
Source code	`src/services/`	Application use cases and orchestration. Services coordinate domain rules, repositories, and external gateways without depending on the delivery mechanism.	`process_invoice.py`, `approve_invoice.py`, `submit_invoice.py`
Source code	`src/agents/`	Agent implementations, tool adapters, role definitions, prompt loaders, context construction, and loop coordination when agent behavior ships as part of the product.	`planner_agent.py`, `coder_agent.py`, `reviewer_agent.py`
Source code	`src/models/`	Domain entities, value objects, data-transfer models, schemas, and important invariants. Keep persistence models separate when needed.	`invoice.py`, `vendor.py`, `validation_result.py`
Source code	`src/utils/`	Small reusable utilities that serve several modules. Guard against turning this directory into a dumping ground.	Date normalization, checksum generation, structured logging helpers.
Source code	`src/config/`	Configuration loading, environment validation, feature flags, typed settings, dependency wiring. Never hard-code actual secrets here.	`settings.py` requiring `ACCOUNTING_API_KEY` from the environment.
Tests	`tests/unit/`	Fast, isolated tests for functions, classes, domain rules, and error handling. Focused fakes or stubs replace external services.	Tax calculation, invoice-number normalization, inactive-vendor rejection.
Tests	`tests/integration/`	Tests across real internal boundaries such as service-to-database, serialization, queue behavior, or sandbox APIs.	Store an invoice in a test database and retrieve the complete aggregate.
Tests	`tests/acceptance/`	End-to-end scenarios demonstrating that user or business requirements hold. Often traceable directly to requirement IDs.	Upload a valid invoice and verify that it reaches the approved state.

Type	File or Directory	Typical Contents	Example
Tests	`tests/regression/`	Tests created specifically to keep previously discovered defects from returning.	A decimal rounding example that once produced an incorrect total.
Tests	`tests/fixtures/`	Stable test inputs, builders, sample documents, fake responses, expected outputs. Never store real customer secrets or production data.	Sanitized invoice PDFs and accounting API response samples.
Scripts	`scripts/build.sh`	A repeatable build procedure with strict error handling. Installs or verifies dependencies, compiles code, creates packages, produces checksums.	Build the application image and place release artifacts in `dist/`.
Scripts	`scripts/test.sh`	The canonical test entry point for humans, CI, and agents. Sets up the test environment and returns a nonzero exit code when verification fails.	Run unit tests, integration tests, and coverage checks.
Scripts	`scripts/lint.sh`	Formatting, linting, type checking, static analysis, and repository policy checks, with check-only and auto-fix modes.	Run formatter verification, linter, type checker, and secret scanner.
Scripts	`scripts/release.sh`	Packaging and release automation with safeguards. Verifies a clean repository, derives the version, builds artifacts, creates tags, publishes after approval.	Refuse to release when tests fail or the changelog lacks the target version.
Scripts	`scripts/bootstrap.sh`	One-command local setup. Checks prerequisites, creates local environments, installs dependencies, generates safe local configuration, states the next command.	Create `.venv`, install dependencies, copy `.env.example` to `.env` without overwriting an existing file.
Generated output	`output/generated-code/`	Temporary or candidate code an agent produced before review and promotion into the authoritative source tree. Many teams skip this directory and let agents edit `src/` directly.	An experimental implementation awaiting review.
Generated output	`output/generated-docs/`	Generated documentation previews, reports, converted formats, or drafts that are not yet authoritative.	Generated HTML API reference or a draft requirements report.
Generated output	`output/reports/`	Test, coverage, security, performance, review, dependency, and traceability reports. Regenerated rather than hand-edited.	`coverage.xml`, `security-scan.json`, `review-summary.md`
Generated output	`output/releases/`	Built release packages, archives, manifests, checksums, and software bills of materials.	`invoice-service-1.2.0.tar.gz`, checksum file, and SBOM.
Generated output	`logs/`	Local application and agent execution logs. Logs may hold sensitive operational data and generally stay out of version control.	`agent-loop.log`, `application.log`, and test-run diagnostics.
Generated output	`temp/`	Disposable scratch files, intermediate conversions, downloaded test assets, lock files, and work in progress. Nothing here is authoritative.	Extracted PDF pages or temporary generated prompts.
Secrets	`secrets/README.md`	Safe instructions for obtaining, naming, storing, rotating, and injecting secrets. Holds no keys. Because the rest of `secrets/` is ignored, `.gitignore` needs an exception to commit this file.	"Copy approved development credentials from the organization's secret manager. Never paste production credentials into this directory."
Secrets	`secrets/local.env`	Actual developer-local credentials and sensitive settings. Excluded from Git and protected by filesystem permissions. Prefer a real secret manager for shared or production use.	`ACCOUNTING_API_KEY=actual-local-value`. Never place the real value in documentation or chat transcripts.

Notes on the Inventory

The original structure diagram named the `prompts/planning/`, `prompts/specification/`, `prompts/implementation/`, `prompts/testing/`, `prompts/documentation/`, and `prompts/release/` directories without listing individual files. It also named the directories under `src/` without source filenames. The paths in those groups are a suggested concrete set rather than a fixed requirement; rename them to match your orchestration tool.

Workflow definitions appear here as YAML. That format is an example only. Your orchestrator may require JSON, TOML, Python, or another schema.

Two files carry more weight than the rest. `AGENTS.md` governs agent behavior on every iteration, and `ROADMAP.md` sets the outcome sequence that planning prompts consume. Both appear below at full length.

Appendix B: Expanded `AGENTS.md` Example

```
# Agent Instructions

## Mission

Implement the behavior defined by the approved project requirements and
specifications while preserving security, maintainability, and traceability.

## Required Reading

Before changing code, read:

1. PROJECT.md
2. docs/requirements.md
3. docs/specification.md
4. docs/architecture.md
5. Relevant files in docs/decisions/
6. state/current-task.json

Instructions in a more specific nested AGENTS.md override this file for
files inside that directory.

## Source of Truth

Use the following order of authority:

1. Approved requirements
2. Approved specification
3. Architecture decisions
4. Existing automated tests
5. Existing implementation
6. Agent assumptions

When two higher-authority artifacts conflict, stop and record the conflict.
Do not silently choose one.

## Work Rules

- Work on only the active task.
- Make the smallest complete change that satisfies its acceptance criteria.
- Do not add unrelated features or broad refactoring.
- Do not change public interfaces unless the specification requires it.
- Do not modify approved requirements to make the implementation easier.
- Record important new architecture decisions in an ADR.
- Never write credentials, tokens, private keys, or customer data to the repo.
- Never disable a test merely to obtain a passing result.

## Required Verification

Before reporting completion, run:

    ./scripts/lint.sh
    ./scripts/test.sh

Also run any task-specific verification listed in state/current-task.json.

## Test Expectations

- Add or update tests for every behavior change.
- Include negative and boundary cases.
- Add a regression test for every defect fix.
- Prefer testing observable behavior over private implementation details.
- Do not claim a test passed unless the command was actually executed.

## Allowed Changes
```

```
The implementation agent may normally modify:

- src/
- tests/
- Relevant documentation
- state/iteration-summary.md

The implementation agent must not modify without explicit task authorization:

- PROJECT.md
- Approved requirement identifiers
- Existing ADR decisions
- Release artifacts
- CI security policies
- Secret files

## Handling Ambiguity

When a minor implementation detail is unspecified:

1. Follow established repository patterns.
2. Choose the least surprising behavior.
3. Record the assumption in the iteration summary.
4. Add a test demonstrating the chosen behavior.

When ambiguity materially affects user behavior, security, data, or public
interfaces, do not guess. Mark the task blocked and describe the required
decision.

## Completion Report

At the end of an iteration, update state/iteration-summary.md with:

- Task identifier
- Brief implementation summary
- Files changed
- Tests added or changed
- Commands executed
- Results
- Assumptions
- Remaining risks or blockers
- Recommended next action

A task is complete only when all acceptance criteria and required checks pass.
```

Appendix C: Expanded `ROADMAP.md` Example

```
# Product Roadmap

## Guiding Outcome

Reduce manual invoice-processing effort by at least 80% while maintaining
an auditable approval trail.

## Phase 1: Reliable Intake

### Outcome
Users can upload supported invoices and receive a normalized invoice record.

### Major capabilities
- PDF and image upload
- Text and field extraction
- File-type and size validation
- Audit record creation
```

```
### Exit criteria
- At least 95% of the reference invoice set is parsed successfully.
- Unsupported files receive actionable error messages.
- Every upload produces an audit event.

### Dependencies
- Sample invoice corpus
- Document-processing service selection

## Phase 2: Business Validation

### Outcome
The system identifies invoices that are valid, incomplete, duplicated, or
in conflict with business policy.

### Major capabilities
- Vendor validation
- Duplicate detection
- Total and tax verification
- Approval-routing rules

### Exit criteria
- Every approved business rule has an automated acceptance test.
- False-positive duplicate rate is below the agreed threshold.

## Phase 3: Accounting Integration

### Outcome
Approved invoices can be submitted safely to the accounting system.

### Major capabilities
- Authenticated API integration
- Idempotent submission
- Retry and outage handling
- Submission-status tracking

### Exit criteria
- Duplicate submissions cannot create duplicate accounting records.
- Failed submissions can be safely retried.
- Integration tests pass against the accounting sandbox.

## Phase 4: Human Review

### Outcome
Reviewers can inspect, correct, approve, or reject exceptional invoices.

### Major capabilities
- Review queue
- Field correction
- Approval history
- Operational reporting

## Deferred Items

The following are not currently scheduled:

- Mobile-native application
- Replacement of the accounting platform
- Automated payment execution
- Support for handwritten invoices

## Roadmap Risks

- Extraction quality may vary significantly across vendors.
- Accounting API rate limits are not yet confirmed.
- Approval rules may differ between business units.
```

The roadmap describes major outcomes over time. Detailed, machine-actionable work belongs in `state/backlog.json` so that the roadmap does not grow into a 2,000-line task list.